Pricer d'options par arbres recombinants en Python*

Doryan Lochet† Cloé Legrand‡

Octobre 2026

Abstract

Notre pricer value des options européennes et américaines par arbre binomial ou trinomial, avec dividendes discrets, pruning, deux constructions (forward et 100 % backward), la méthode BBS et le calcul des grecques. Il se pilote depuis Excel. Nous montrons que le pruning divise le temps de calcul par 5 pour un écart de prix inférieur à 10^{-7} €, et que la construction backward divise la mémoire par 70 mais double le temps. Le trinomial BBS est environ 600 fois plus précis que le binomial BBS. Enfin, pour les grecques, un choc trop petit donne du bruit et un choc trop grand un biais.

*Binomial, trinomial, dividendes discrets, pruning, méthode BBS et grecques. Projet du cours d'informatique quantitative, M2 EIF.

†M2 EIF, Informatique quantitative

‡M2 EIF, Informatique quantitative

1 Présentation du pricer

Fonctionnement

Tout se fait depuis Excel, en trois étapes :

— remplir les cases jaunes de l'onglet **Parametres** de *parametres_pricer.xlsx* (marché, option, dates, N ou précision visée, type d'arbre, BBS, pruning, dividendes…) ;

— enregistrer puis **fermer** le fichier ;

— lancer la cellule `run_from_excel` de *main.ipynb*.

Les résultats (prix, écart avec Black-Scholes, grecques, infos sur l'arbre) s'écrivent dans l'onglet **Resultats**, le dessin dans **Visualisation**.

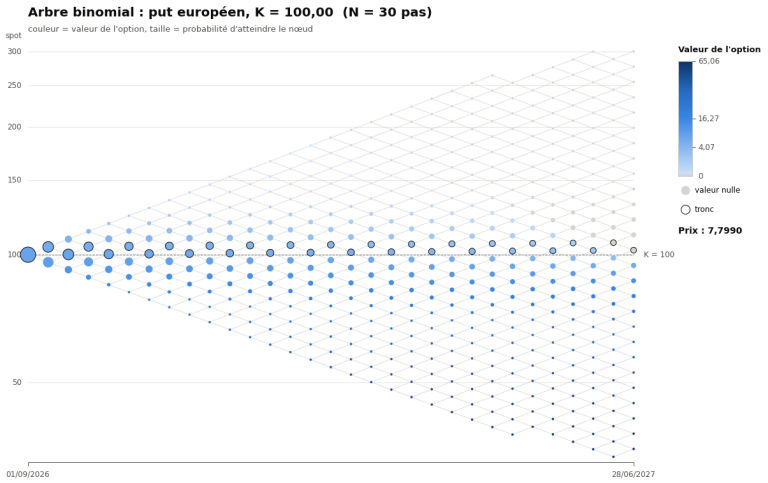


Figure 1 : arbre binomial de 30 pas, put européen ($S_0 = K = 100$, $r = 3\,\%$, $\sigma = 25\,\%$, $T \approx 0{,}82$ an)

Axe en échelle log (nœuds régulièrement espacés), couleur = valeur de l'option, taille = probabilité d'atteindre le nœud. En binomial, le tronc (cerclé) alterne haut et bas. Le prix converge : **7,7990 €** à 30 pas, **7,7489 €** à 1 000 pas, pour **7,7475 €** en Black-Scholes.

Code et validité

Quatre fichiers : `tree.py` (arbre, Black-Scholes, grecques), `excel_interface.py`, `visualisation.py` et `etudes.py` (les 12 graphiques, via `generate_all()`). Trois librairies : *openpyxl*, *matplotlib*, *Pillow*. On parcourt l'arbre uniquement par les liens entre nœuds, sans liste. Nos contrôles :

— prix d'un call de strike 0 = prix forward du sous-jacent ;

— forward et backward donnent **exactement** le même prix ;

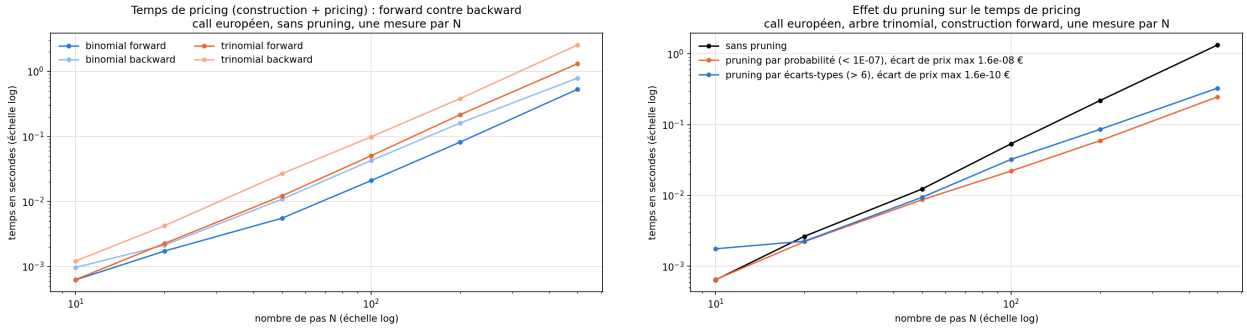
— la parité call-put est respectée à $4 \cdot 10^{-12}$ près ;

— sans dividende, call américain = call européen ; put américain > put européen.

2 Principaux résultats

Cadre des graphiques : call européen à la monnaie, $S_0 = K = 100$, $r = 3\,\%$, $\sigma = 25\,\%$, $T = 1$ an, sans dividende.

2.1 Temps de calcul et mémoire



(a) temps de pricing, forward contre backward

(b) effet du pruning sur le temps de pricing (trinomial, forward)

Figure 2 : temps de calcul

— **Le forward est 1,5 à 2 fois plus rapide** : le backward construit chaque nœud deux fois. Il refait d'abord tout le passage aller (`build_trunk`, le seul à donner les probabilités du pruning), puis reconstruit chaque colonne en revenant. Le pricing lui-même est négligeable (0,05 s sur 0,6 s à $N = 400$).

— **Le pruning** (une seule branche pour un nœud de probabilité $< 10^{-7}$, ou à plus de 6 écarts-types du tronc) divise le temps par 4 à 5 à $N = 500$, pour un écart de prix d'au plus $1{,}6 \cdot 10^{-8}$ €.

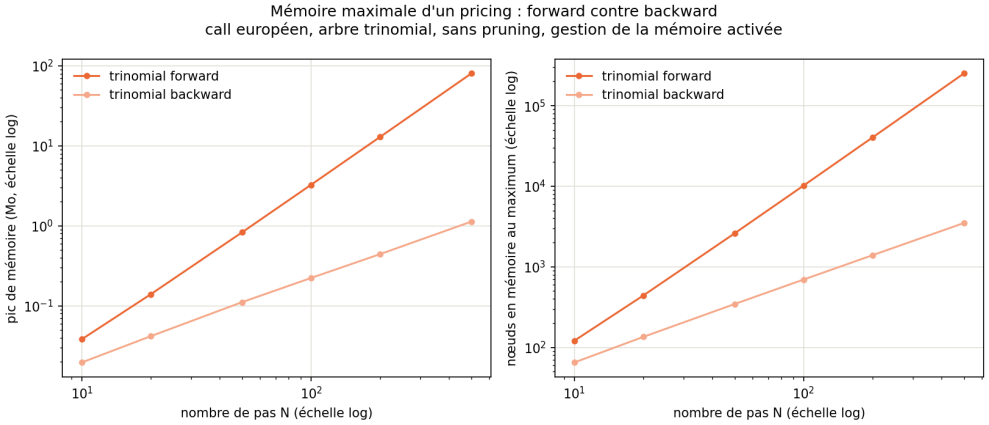


Figure 3 : pic de mémoire, forward contre backward (en Mo et en nombre de nœuds)

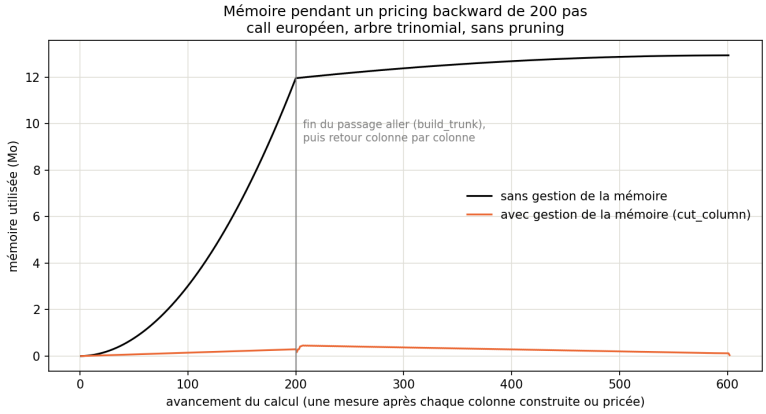


Figure 4 : mémoire d'un pricing backward de 200 pas, avec et sans gestion de la mémoire

— **Le backward est 70 fois plus économe** : il n'a jamais plus de 2 colonnes en mémoire (1 Mo à $N = 500$, croissance en N), contre tout l'arbre en forward (80 Mo, croissance en N^2). C'est possible parce que pricer une colonne ne demande que la colonne suivante. Le forward, lui, doit avoir construit tout l'arbre avant de commencer à pricer depuis la maturité. D'où un arbitrage entre temps et mémoire.

— **Ce gain suppose de libérer les nœuds à la main** : ils se pointent mutuellement (`up` ↔ `down`) et restent reliés au tronc, donc Python ne les supprime jamais. `cut_column` remet ces liens à `None` dès qu'une colonne ne sert plus, et ses nœuds sont aussitôt libérés. Sans `cut_column`, le backward garde tout l'arbre (12 Mo, comme le forward) ; avec, il reste sous 0,5 Mo.

— **La gestion de la mémoire n'a donc pas le même effet selon la méthode.** En backward, elle supprime chaque colonne dès qu'elle ne sert plus, pendant le calcul : c'est elle qui fait passer de 12 Mo à moins de 0,5 Mo. En forward, elle ne baisse pas le pic, car tout l'arbre est nécessaire jusqu'à la fin du pricing ; `release()` ne libère l'arbre qu'une fois le prix calculé. Elle sert alors quand on enchaîne plusieurs arbres, comme les 7 arbres des grecques : chacun est libéré tout de suite au lieu de s'accumuler.

2.2 Le pruning codé en binaire

En backward, on reconstruit l'arbre en partant de la maturité. Or, pour savoir si un nœud doit être élagué, il faut sa probabilité d'atteinte, qui ne se calcule qu'en avançant depuis la racine. Il faut donc noter pendant le passage aller quels nœuds ont été élagués, puis relire cette information au retour.

Pour cela, on écrit un nombre binaire par colonne, avec un chiffre par nœud : **1 si le nœud est élagué, 0 sinon**.

— **Exemple** : `0110` signifie que les 2e et 3e nœuds sont élagués, et les 1er et 4e non.

— **À l'aller**, on met à 1 le chiffre de chaque nœud élagué (`_link_side`).

— **Au retour**, on relit ce chiffre pour savoir si le nœud était élagué (`_was_pruned`).

Un seul entier suffit donc pour toute une colonne, quelle que soit sa taille.

Le binaire, comment ça marche ? En binaire, on n'utilise que deux chiffres, 0 et 1. On lit de droite à gauche, et chaque position vaut le double de la précédente : 1, 2, 4, 8… Le chiffre le plus à droite correspond au 1er nœud, celui d'à côté au 2e, et ainsi de suite.

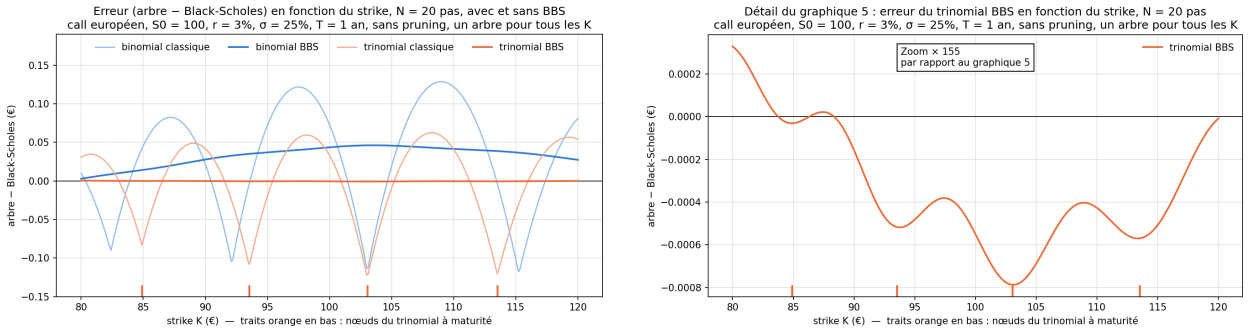
— **Du binaire vers un nombre** : on additionne les positions qui valent 1. $0110 = 0 \times 8 + 1 \times 4 + 1 \times 2 + 0 \times 1 = 6$.

— **D'un nombre vers le binaire** : on divise par 2 et on garde les restes, lus de bas en haut. $6 \div 2 = 3$, reste 0 ; $3 \div 2 = 1$, reste 1 ; $1 \div 2 = 0$, reste 1, ce qui donne `110`, soit `0110`.

Python stocke donc simplement l'entier 6, et ce nombre contient à lui seul toute l'information de la colonne.

Pourquoi cette méthode ? Elle respecte la consigne de ne pas utiliser de liste. Avec une liste des nœuds élagués, il faudrait la parcourir pour savoir si un nœud y figure, c'est-à-dire faire une boucle de plus. Ici, la réponse s'obtient en une seule opération. Elle est aussi très compacte : 1 bit par nœud, stocké sur le nœud du tronc.

2.3 Mesure de l'erreur



(a) erreur selon le strike, $N = 20$, avec et sans BBS

(b) zoom sur le trinomial BBS (×155 par rapport à la figure 5a)

Figure 5 : erreur (arbre − Black-Scholes) en fonction du strike

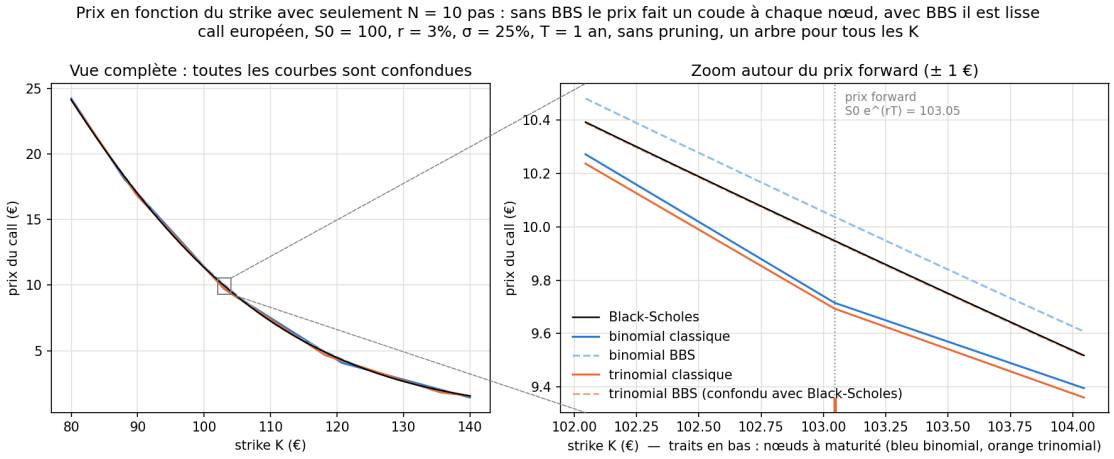


Figure 6 : prix selon le strike avec 10 pas, zoom autour du forward

Ce qu'on voit : sans BBS, l'erreur oscille entre $-0{,}12\,€$ et $+0{,}13\,€$, avec un creux vers le bas chaque fois que le strike tombe sur un nœud à maturité ; avec BBS, la courbe devient lisse, et presque nulle en trinomial.

Pourquoi l'arbre sous-estime le prix quand le strike tombe sur un nœud ? L'arbre ne connaît qu'un nombre limité de prix finaux, les nœuds : il regroupe sur chaque nœud toute la probabilité de son voisinage. En réalité, l'action peut finir juste au-dessus de K, et l'option rapporte alors un petit peu. Mais si K tombe pile sur un nœud, tous ces scénarios « juste au-dessus de K » sont rangés sur ce nœud, qui rapporte exactement 0. L'arbre oublie donc ces petits gains, et son prix est trop bas. Entre deux nœuds, c'est l'inverse, d'où les arches au-dessus de zéro. La cassure tombe

au prix forward (103,05 €) car c'est la position du nœud final du tronc, dans les deux arbres : le trinomial suit le forward à chaque pas, et le binomial y revient quand N est pair.

Ce que change BBS : sur le dernier pas, au lieu de regarder seulement les nœuds finaux, on utilise la formule de Black-Scholes, qui prend en compte tous les prix possibles autour de K, et pas seulement les nœuds. Les petits gains juste au-dessus de K sont donc comptés : le coude du payoff est lissé et les creux disparaissent.

Pourquoi ça marche si bien en trinomial ? Une fois le dernier pas corrigé, il reste l'erreur des pas précédents, qui dépend de la ressemblance entre un pas de l'arbre et la vraie loi (une courbe en cloche). Avec deux branches, le binomial ne reproduit que le centre et la largeur de la cloche. Avec trois branches (probabilités 1/6, 2/3, 1/6), le trinomial reproduit aussi sa forme exacte, notamment l'épaisseur de ses bords (kurtosis de 3, comme la loi normale, contre 1 en binomial). Chaque pas ressemble donc beaucoup plus à la réalité : l'erreur du trinomial BBS reste sous 0,001 €, alors que le binomial BBS garde un biais de +0,03 à +0,05 €.

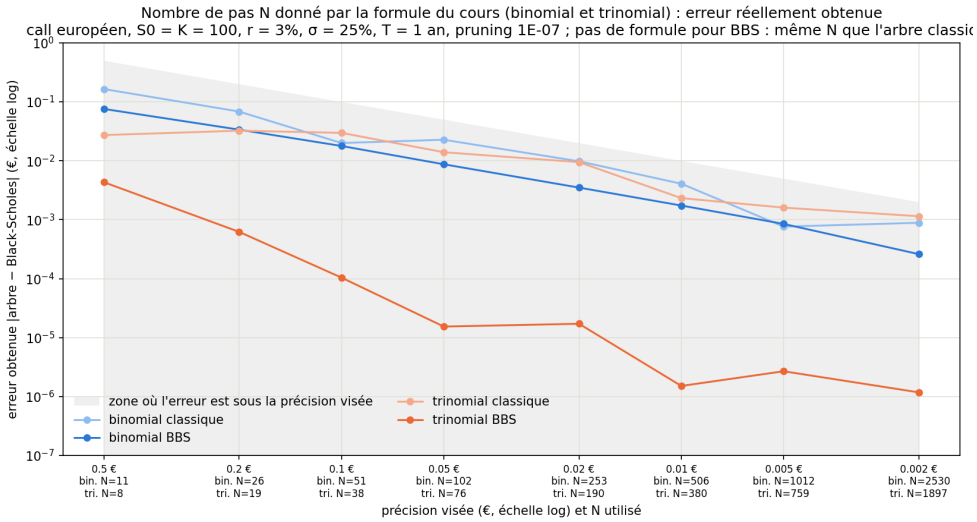


Figure 7 : N donné par la formule du cours pour une précision visée, et erreur obtenue

La zone grise est la zone où l'erreur obtenue est plus petite que la précision visée : un point dans le gris signifie que l'objectif est atteint.

Pour chaque précision visée ε, le pricer calcule le nombre de pas avec la fonction `Model.from_precision` (dans `tree.py`). Elle inverse la formule d'erreur du cours :

$$N = \frac{c \cdot S_0 \, e^{rT} \, \sigma^2 \, T}{\sqrt{2\pi} \; \varepsilon \; \sqrt{e^{\sigma^2 T} - 1}} \tag{1}$$

avec $c = 3/8$ en trinomial et $c = 1/2$ en binomial, car les nœuds ne sont pas espacés de la même façon ($\sigma\sqrt{3\Delta t}$ en trinomial contre $2\sigma\sqrt{\Delta t}$ en binomial). Les N obtenus sont écrits sous l'axe.

Il n'existe pas de formule pour BBS : on reprend donc le même N que l'arbre classique du même type, et on regarde l'erreur obtenue. Les quatre arbres restent dans la zone grise, donc la formule est prudente. Avec le même N, le trinomial BBS est même 100 à 1 000 fois sous la cible.

2.4 La méthode BBS pour une option américaine

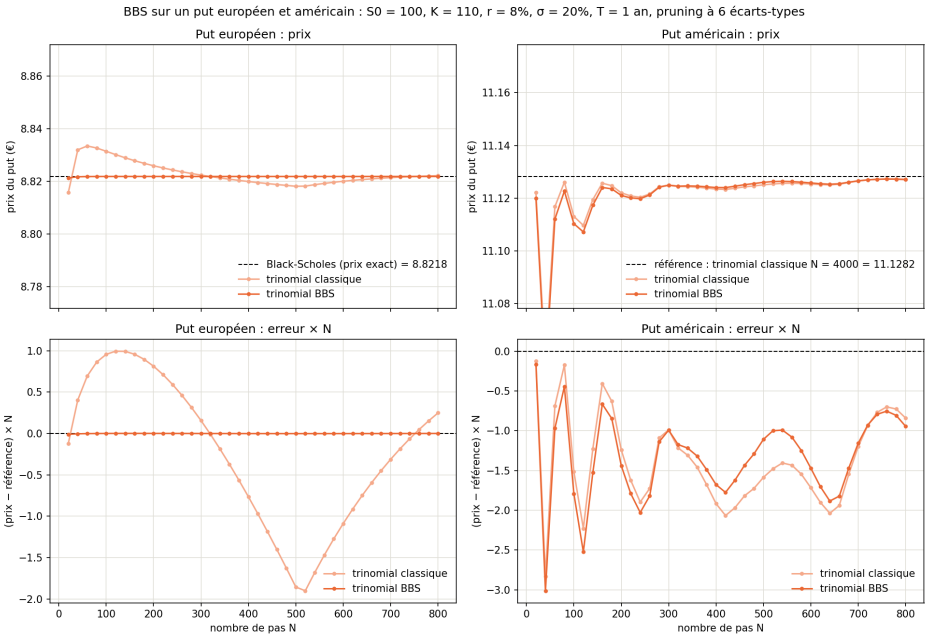


Figure 8 : BBS sur un put européen et américain ($K = 110$, $r = 8\,\%$, $\sigma = 20\,\%$)

— **Option européenne : le trinomial BBS est presque parfait.** Dès $N = 20$, son prix est collé à Black-Scholes (8,8218 €) et son erreur $\times N$ reste à 0, alors que le trinomial classique oscille entre -2 et $+1$.

— **Option américaine : BBS n'apporte presque rien.** Le trinomial BBS suit les mêmes oscillations que le trinomial classique, avec une erreur $\times N$ entre -3 et $-0{,}2$, et les deux sous-estiment le prix de référence (11,1282 €).

Pourquoi ? À cause de l'exercice anticipé. Pour une européenne, toute l'erreur vient du coude du payoff à maturité, que l'arbre ne voit qu'aux nœuds ; BBS lisse justement ce dernier pas, donc il corrige tout. Pour une américaine, on peut exercer à chaque date : l'arbre compare à chaque nœud la valeur de l'option et le gain d'un exercice immédiat. Il existe donc un prix seuil en dessous duquel on exerce, et ce seuil existe **à toutes les dates**, pas seulement à maturité. L'arbre ne le voit qu'aux nœuds, donc en escalier, et BBS ne corrige que le dernier pas. De plus, l'arbre ne peut exercer qu'aux N dates de la grille, et non à tout instant : il manque des occasions d'exercer, ce qui explique qu'il **sous-estime** le prix américain.

2.5 Les grecques et la taille du choc

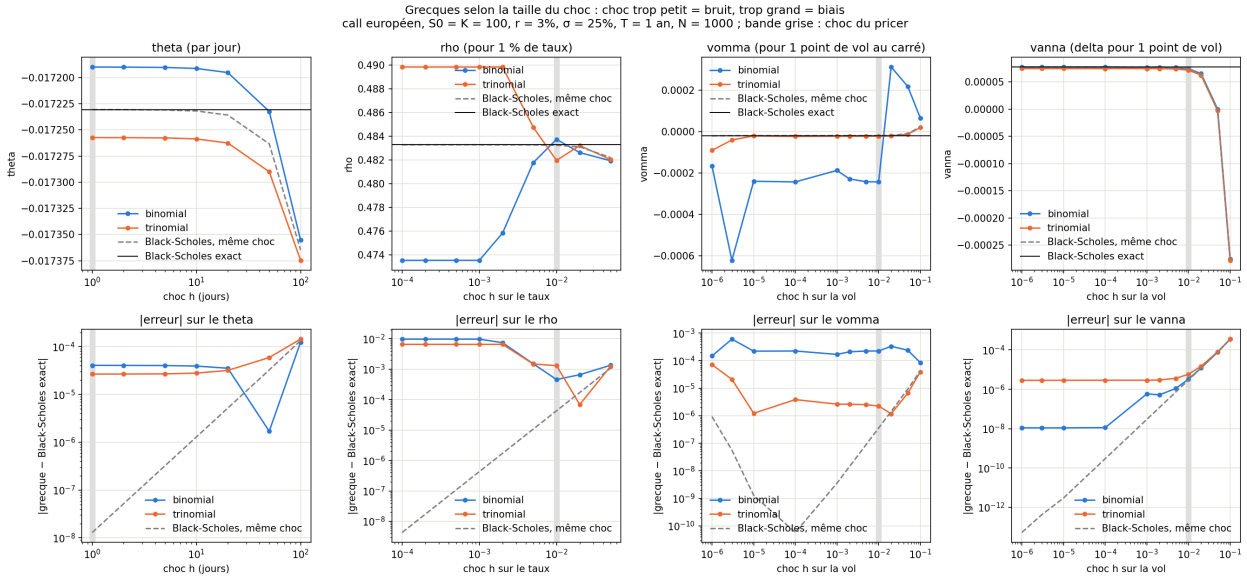


Figure 9 : grecques et erreur selon la taille du choc h ($N = 1\,000$)

Le delta et le gamma se lisent sur l'arbre. Les autres grecques repricent avec un paramètre décalé d'un choc h, grâce à `OneDimDerivative`, à qui l'on passe une fonction (prix en fonction de la vol, du taux ou de la date). En trinomial, les arbres à $\sigma \pm h$ gardent la grille de l'arbre de base (`grid_vol`) : seules les probabilités changent, sinon la vomma devient très bruitée.

Ce graphique sert à **choisir la taille du choc** h. Un choc trop grand donne un **biais** : la différence finie elle-même est fausse, et les courbes rejoignent la courbe grise. Un choc trop petit donne du **bruit** : on mesure les petites irrégularités du prix de l'arbre, et l'erreur ne diminue plus. Le bon choc se trouve entre les deux, au creux de la courbe.

Les chocs de notre pricer (1 jour pour le thêta, 1 % pour le rho, 1 point de vol pour la vomma et la vanna) tombent dans cette bonne zone. Le binomial, dont la grille bouge avec la vol, a en revanche une vomma fausse d'un facteur 10 quel que soit le choc : pour la vomma, il faut utiliser le trinomial.

3 Difficultés rencontrées

— **Savoir, en backward, quels nœuds ont été prunés.** Les probabilités n'existent qu'en avançant dans le temps. Nous gardons l'information du passage aller grâce au codage binaire (partie 2.2).

— **La vitesse dépend de la version de Python** : environ 2 s par option en Python 3.10, contre 0,5 s en 3.13, dont l'interpréteur est bien plus rapide sur un code qui crée beaucoup de petits objets.

— **Des probabilités trinomiales négatives**, toujours sur des nœuds très éloignés du tronc (surtout après un dividende, qui pèse lourd sur un spot très bas). Ce sont justement les nœuds élagués par le pruning : avec lui, le problème n'arrive presque plus.

— **Excel** : *xlwings* plantait ou figeait Excel sur nos deux PC. Nous l'avons remplacé par *openpyxl*, plus simple et stable, mais qui impose de fermer le fichier.

— **L'image de l'arbre** : le SVG est trop lourd et ne s'insère pas dans Excel. Nous y mettons un PNG et gardons le SVG à côté pour zoomer.

— **La mémoire** : il a fallu casser les cycles entre nœuds (`cut_column`). Compter les nœuds via `__del__` double le temps, donc ce comptage n'est activé que pour les mesures.

Conclusion

Il n'y a pas de réglage universel : le forward est rapide, le backward économe en mémoire, le pruning presque gratuit. Le **trinomial BBS** donne de loin la meilleure précision, sauf pour une américaine. Pour les grecques, la taille du choc est un compromis entre bruit et biais.